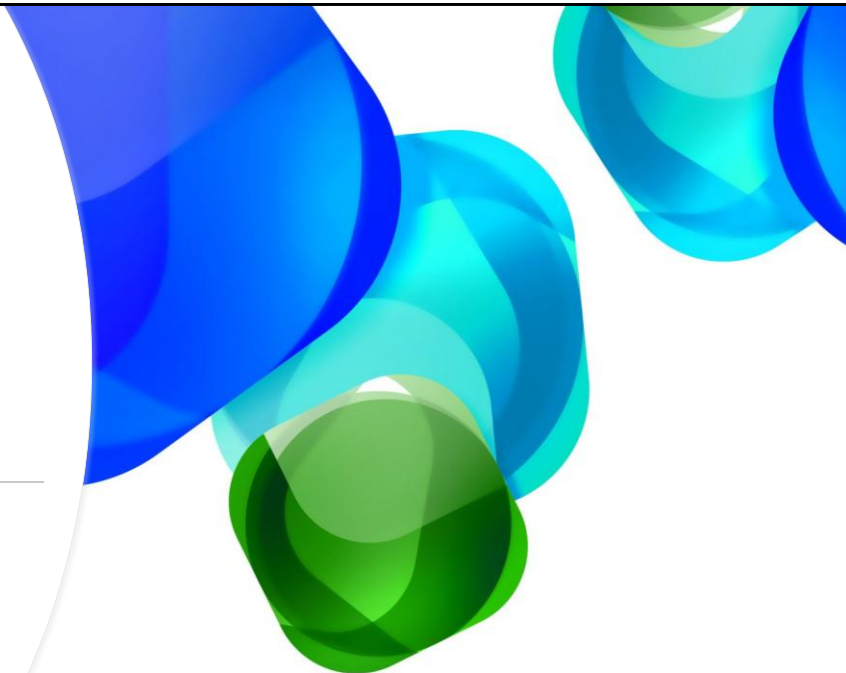




Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak



1

Dogodkovno vodena arhitektura

2

Kaj je EDA?

=pristop k izgradnji poslovne arhitekture, ki omogoča, da sistemi "slišijo" dogodke in se na njih odzovejo inteligentno.

- **Event-Driven Architecture**

- Arhitekturni slog, kjer aplikacije **komunicirajo prek dogodkov** (*event messages*).
- **Producer** objavi dogodek (npr. *OrderCreated*), **consumerji** ga asinkrono obdelajo.
- Dogodki se prenašajo prek **event bus / message brokerja** (npr. Kafka, RabbitMQ, NATS).
- Prednosti:
 - ohlapna sklopljenost,
 - skalabilnost,
 - odzivnost,
 - boljša integracija mikroservisov.

3

Dogodek

- Temeljni gradnik dogodkovno vodenih arhitektur (EDA).

»Relevantna sprememba stanja v sistemu ali njegovem okolju.«

- Dogodek opisuje, da se je nekaj zgodilo (preteklik) in lahko sproži obdelavo pri več porabnikih (consumerjih).

4

Dogodek

- Dogodek je sestavljen iz **glave (metadata)** in **telesu (payload)**.
- **Glava** vsebuje npr.:
 - **ID dogodka**,
 - **tip/ime dogodka**,
 - **časovni žig**,
 - **zaporedno številko/ponovitev (npr. retry/sequence)** ter
 - podatke o **izvoru/iniciatorju**.
- **Telo** vsebuje **podatke o dogodku** (dejanska vsebina, ki jo porabniki obdelajo).

5

Dogodkovno vodena arhitektura (EDA) – ključne ideje

- Sistemi ne kličejo drug drugega neposredno, temveč **reagirajo na objavljene dogodke**.
- Osnovni vzorec: **producer** → **broker/event bus** → **consumer(ji)** (asinkrono).
- Ključni arhitekturni odločitvi:
 - **kateri dogodki obstajajo** in
 - **kako jih poimenujemo/strukturiramo** (event schema, verzioniranje).
- Tipični izzivi:
 - **eventual consistency**,
 - **podvajanje dogodkov (idempotentnost)**,
 - **urejanje zaporedja (ordering)**.

6

Dogodkovno-usmerjeno programiranje

- V praksi pogosto uporabljamo pristope, ki so sorodni EDA (npr. OS, GUI, omrežja ...).
- **EDP (Event-Driven Programming)** je koncept, kjer izvajanje programa sprožijo **dogodki** in **obravnavalniki**.
- **EDP vključuje:**
 - **pošiljatelja/razpošiljevalnika dogodkov (event dispatcher)**
 - vzdržuje seznam registriranih poslušalcev (*subscriptions*) za posamezne tipe dogodkov
 - **poslušalce / obravnavalnike dogodkov (event listener / handler)**
 - deli kode, ki se sprožijo ob določenem dogodku
 - pri dispatcherju se **registrirajo** na izbrane dogodke

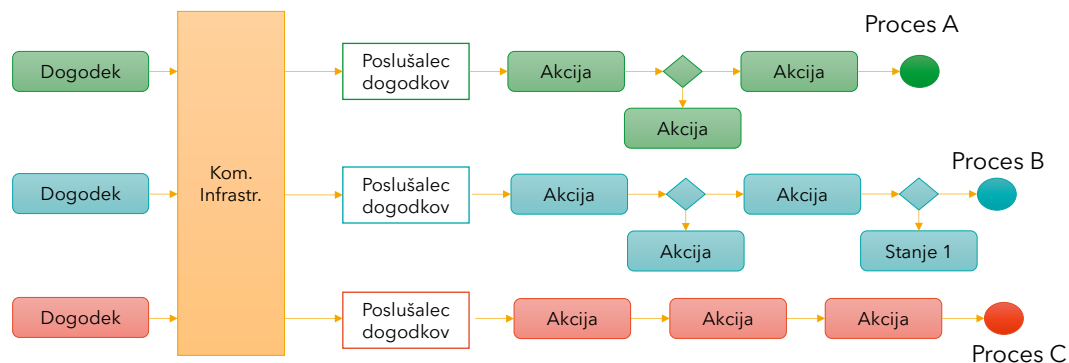
7

Dogodkovno-usmerjeno programiranje

- **Razlika od tradicionalnega (zaporednega) programiranja:**
 - potek izvajanja ni vnaprej določen v osrednji programski logiki, temveč ga **sprožajo dogodki**.
- Rešitev praviloma nima enega osrednjega dela, ki bi vodil celoten potek, ampak se ob dogodkih izvajajo **prijavljeni obravnavalniki dogodkov**.

8

Dogodkovno-usmerjeno programiranje



V dogodkovno-usmerjeni rešitvi poslušalci dogodkov prejmejo informacijo o spremembi stanja (dogodku) in jih posredujejo do komponent, ki aktivirajo procese, ki temeljijo na dogodkih.

Kot je prikazano na sliki, poslušalci dogodkov se lahko izvedejo sočasno.

9

EDA : EDP

• Podobnosti

- Temelji na **dinamičnem povezovanju** delov sistema.
- **Komunikacija s sporočili** omogoča **šibko povezanost** in **nesočasno** delovanje.
- Deli sistema se povežejo na skupni prenosni kanal in se **prijavijo na vrste dogodkov**, ki jih zanimajo.

• Glavna razlika

- Pri dogodkovno usmerjenem programiranju se običajno obravnavajo **nizkonivojski dogodki** (npr. klik miške, pritisk tipke).
- Pri dogodkovno vodeni arhitekturi se običajno obravnavajo **visokonivojski dogodki**, povezani z delovanjem sistema (npr. oddano novo naročilo, potrjeno plačilo).

10

Osnovne komponente EDA

Generatorji dogodkov

- Avtonomne komponente (npr. senzorji, bralniki, analitični algoritmi), ki spremljajo dogajanje na določenem področju in ob spremembi ustvarijo dogodek.
- Primeri: temperatura prostora, zasedenost diska, spremljanje bančnih transakcij, poraba energije v hiši.

Poslušalci (odjemalci) dogodkov

- Povezani na skupno vodilo in čakajo na dogodek določene vrste.
- Ko dogodek prejmejo, sprožijo nadaljnje izvajanje; posledica so lahko tudi novi dogodki.

Procesne enote dogodkov

- Komponente, ki dogodke obdelajo: jih preverijo, dopolnijo, preoblikujejo, združijo ali posredujejo naprej.

11

Generatorji dogodkov

- Dogodkovno vodena arhitektura ne deluje, če **ne zaznava in ne prejema dogodkov**.
- Da dogodki nastanejo, jih je treba **ustvariti** in **poslati** komponentam, ki jih obravnavajo.
- Generatorji dogodkov so komponente, ki **ustvarjajo dogodke**:
 - najpogosteje **programske komponente**,
 - lahko tudi **strojna oprema**, ki analogni signal pretvori v digitalni zapis, iz katerega program ustvari dogodek.
- Nahajajo se lahko **kjerkoli v arhitekturi**, znotraj ali zunaj organizacije.
- V podatke dogodka običajno vključijo **vse podatke, potrebne za nadaljnjo obdelavo** (čeprav to ni vedno nujno).

12

Poslušalci dogodkov

- Tako kot generatorji dogodkov se lahko nahajajo **kjerkoli v arhitekturi**.
- Med generatorjem in poslušalcem mora biti vzpostavljena **komunikacijska povezava** (na primer prek skupnega vodila).
- Poslušalec dogodkov je praviloma **programska oprema** (lahko tudi strojna), ki zna:
 - **prepoznati dogodke** med prejetimi podatki in
 - ugotoviti, **kdaj gre za dogodek** in kdaj za druge vrste sporočil.
- V najpreprostejši izvedbi poslušalec prejema samo dogodke, ki ga zanimajo.
- V praksi pa mora biti jasno določeno, **katere dogodke posluša** oziroma **na katere vrste dogodkov je prijavljen**.

13

Procesne enote dogodkov

- Procesna enota dogodkov je **programska komponenta**, ki dogodke **obdeluje** ter na podlagi pravil določi **pomen dogodka** in **naslednja dejanja**.
- **Vrste obdelave dogodkov**
 - **Preprosta obdelava**
 - vsak dogodek se obravnava posamično; logiko običajno izvajajo odjemalci.
 - **Tokovna obdelava**
 - dogodki se obdelujejo kot tok; upoštevata se zaporedje in časovna bližina dogodkov.
 - **Kompleksna obdelava**
 - obravnavajo se daljše serije dogodkov; na podlagi vnaprej določenih pravil se prepoznajo vzorci in določijo posledična dejanja.

14

Odziv na dogodke

- Po obdelavi dogodka se izvedejo **dejanja**, ki predstavljajo **odziv na dogodek**.
- **Možne oblike odziva**
 - **samodejni odziv v sistemu**,
 - **samodejno obveščanje** drugih sistemov in/ali ljudi,
 - odziv, ki **zahteva posredovanje človeka**,
 - ...
- **Samodejni odziv**
 - izvedba **samodejnih postopkov** (na primer z orkestracijo storitev ali z delovnimi tokovi).
- **Odziv s posredovanjem človeka**
 - vključitev strokovnjakov, kadar je potrebno **razumevanje podatkov dogodka** in njihova **umestitev v potek dela**, v katerem je do dogodka prišlo.

15

Življenjski cikel dogodka

- Generiranje dogodka (iniciator/generator)
- Prenos dogodka
(preko kanala od generatorja do procesne enote dogodkov)
- Procesiranje dogodka
(ovrednotenje informacij v telesu; po potrebi združevanje, filtriranje skladno s poslovnimi pravili; neodvisnost od generatorja)
- Odziv na dogodek
(ko je dogodek zanimiv za kakšnega odjemalca)

16

Vrste EDA

- 2 osnovna tipa EDA (Manas Deb, Oracle Fusion):
 - **EKSPPLICITNA EDA**
 - **IMPLICITNA EDA**
- Eksplicitna EDA
 - Generatorji dogodkov vedo kdo so poslušalci in komu pošiljajo dogodke
 - **Tesna sklopljenost** generatorji : poslušalci
- Implicitna EDA
 - Ne definira povezav med generatorji in poslušalci dogodkov
 - Poslušalci dogodkov ter procesne enote dogodkov same določijo kateri dogodki jih zanimajo
 - **Šibka sklopljenost** generatorji : poslušalci
 - Omogoča razvoj bolj dinamičnih oz. agilnih rešitev

17

SOA + EDA

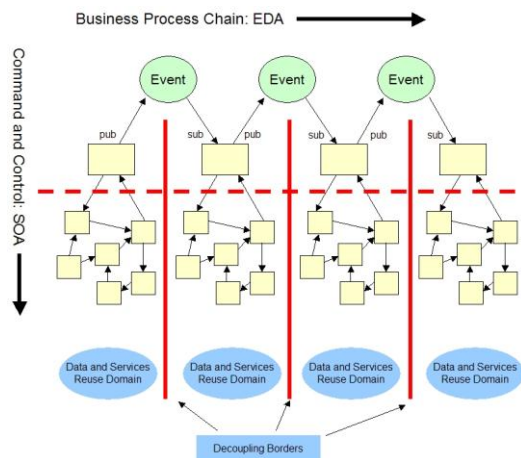
- Povezavo med storitveno usmerjeno in dogodkovno vodeno arhitekturo najlažje razumemo v okviru **upravljanja poslovnih procesov**.
- Sodobna orodja za upravljanje poslovnih procesov pogosto temeljijo na **jeziku za izvajanje poslovnih procesov (BPEL)**.
- BPEL je po zasnovi bližje **storitveno usmerjeni arhitekturi**, ker omogoča **vodenje in povezovanje storitev** v zaporedje korakov (postopkovni pristop).
- Poleg tega podpira tudi **opis sodelovanja** med več udeleženci procesa, kar je bližje **dogodkovno vodeni arhitekturi**.
- BPEL je večinoma **postopkovno** naravnani, ne pa **opisno**.
- Razvoj pristopov pri opisu procesov gre v smer **opisnega (deklarativnega) pristopa**.

18

SOA + EDA

Jack van Hoof, **How EDA extends SOA and why it is important**

<http://soa-eda.blogspot.com/2006/11/how-eda-extends-soa-and-why-it-is.html>



If you are seeking to support independency between business process steps, EDA is the way to go. This style of architecture is appropriate in federated and autonomous processing environments.

19

EDA – povzetek

- Osnovna enota dogodkovno vodenih arhitektur je **dogodek**.
- Dogodkovno vodene arhitekture vsebujejo iniciatorje oz. generatorje dogodkov in prejemnike oz. odjemalce dogodkov.
- Dogodkovno vodena arhitektura je **šibko sklopljena**.
- Dogodkovno vodene arhitekture uporabljajo metode **asinhronega komuniciranja**,
 - običajno mehanizme, ki uporabljajo vzorec **Objavi/Naroči** (Publish/Subscribe).
- Dogodkovno vodene arhitekture uporabljajo splošno dostopne **komunikacijske sisteme**,
 - storitvena vodila (ESB), adapterje in mediatorje, ki omogočajo prenos sporočil.
- Dogodkovno vodena arhitektura **ne temelji na centralnem kontrolerju!**

Hugh Taylor, Angela Yochem, Les Phillips, and Frank Martinez, *EVENT-DRIVEN ARCHITECTURE, How SOA enables the Real-Time Enterprise*, 2009

20

Prednosti EDA za poslovno okolje

- **Agilnost** pri operativnem upravljanju sprememb stanj.
- **Napredno vzpostavljanje povezav med podatki**, ki jih lahko podjetje uporabi pri analizi poslovanja, modeliranju poslovnih procesov ali upravljanju.
- **Dinamična vpeljava poslovnih pravil**, ki določajo odzivanje na določene posamezne dogodke ali serije dogodkov.
- **Razvoj inteligentnih rešitev** na osnovi procesiranja podatkov/dogodkov

Hugh Taylor, Angela Yochem, Les Phillips, and Frank Martinez, *EVENT-DRIVEN ARCHITECTURE, How SOA enables the Real-Time Enterprise.*, 2009

21

Cloud-native arhitekture

22

Gartner napoved za prehod na cloud-native arhitekture

- Do leta 2025 bo več kot 85% podjetij razvijalo na osnovi cloud-first principa
- Cilji, povezani z digitalizacijo in digitalnimi strategijami, bodo zahtevali prehod na cloud-native arhitekture rešitev ter povezane tehnologije

"Adopting cloud-native platforms means that digital or product teams will use architectural principles and capabilities to take advantage of the inherent capabilities within the cloud environment,"

"New workloads deployed in a cloud-native environment will be pervasive, not just popular and anything noncloud will be considered legacy."

- Pri organizaciji napovedujejo, da bo v letu 2025 večina (več kot 95%) digitalnih rešitev nameščenih na cloud-native platformah (kar je zelo velik skok iz 30% v letu 2021)

Vir: <https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences>

23

Računalništvo v oblaku

Računalništvo v oblaku je model za omogočanje vseprisotnega, priročnega omrežnega dostopa na zahtevo do skupnega nabora nastavljivih računalniških virov (npr. omrežij, strežnikov, pomnilnika, aplikacij in storitev), ki jih je mogoče hitro zagotoviti in sprostiti z minimalnim naporom upravljanja ali interakcijo s ponudnikom storitev. Ta model oblaka je sestavljen iz petih bistvenih karakteristik, treh modelov storitev in štirih modelov uvajanja.

National Institute of Standards and Technology, 2011

24

Računalništvo v oblaku – bistvene karakteristike

- Samopostrežba storitev na zahtevo (*ang. on-demand self-service*)
- Širokopasovni omrežni dostop (*ang. broad network access*)
- Združevanje ali konsolidacija virov (*ang. resource pooling*)
- Hitra prožnost (*ang. rapid elasticity*)
- Merjena storitev (*ang. measured service*)

National Institute of Standards and Technology, 2011

25

Računalništvo v oblaku – samopostrežba storitev na zahtevo

- Potrošnik si lahko samostojno in avtomatsko zagotovi računalniške zmogljivosti po potrebi
- Računalniške zmogljivosti se lahko določijo glede na
 - strežniški čas,
 - kapacitete shrambe,
 - itd.
- Zmogljivosti se zagotovijo brez zahteve po posredovanju ali interakciji s človekom na strani ponudnika storitev

National Institute of Standards and Technology, 2011

26

Računalništvo v oblaku – širokopasovni omrežni dostop

- Zmožljivosti so na voljo preko omrežja in do njih se dostopa prek standardnih mehanizmov, ki spodbujajo uporabo na heterogenih odjemalcih
- Podprti morajo biti tako tanki kot debeli odjemalci
 - mobilnih naprave,
 - tablice,
 - računalnikov,
 - itd.

National Institute of Standards and Technology, 2011

27

Računalništvo v oblaku – združevanje ali konsolidacija virov

- Najemnikom storitev je omogočeno, da se različni fizični in virtualni viri dinamično dodeljujejo glede na povpraševanje
 - potrošnik v splošnem nima nadzora in/ali znanja o točni lokaciji virov, ki so bili zagotovljeni.
 - lahko se sicer določi lokacija na višji ravni abstrakcije npr. kontinent, država ali podatkovno središče.
- Primeri virov vključujejo:
 - podatkovna shramba,
 - pomnilnik,
 - pasovna širina omrežja,
 - ...

National Institute of Standards and Technology, 2011

28

Računalništvo v oblaku – hitra prožnost

- Zmogljivosti je mogoče prožno zagotoviti in sprostiti
 - v nekaterih primerih se zmogljivosti skalirajo avtomatsko, sorazmerno s trenutnim povpraševanjem.
- Potrošnik ima ob tem pogosto občutek, da ima na voljo neomejeno zmogljivosti

National Institute of Standards and Technology, 2011

29

Računalništvo v oblaku – merjena storitev

- Sistemi v oblaku samodejno nadzirajo in optimizirajo uporabo virov
 - na osnovi nenehnega izvajanja meritev na določenem nivoju abstrakcije, ki je specifičen za tip storitve
- Uporaba virov se spremlja, upravlja in poroča, kar omogoča transparenten pregled stanja tako za ponudnika storitve
- Spremljanje uporabe virov prav tako omogoča transparenten pregled nad aktivnostmi uporabnika storitev v oblaku

National Institute of Standards and Technology, 2011

30

Osnovni storitveni modeli računalništva v oblaku

- Osnovni storitveni modeli oblaka:
 - Programska oprema kot storitev (ang. *Software as a Service* - SaaS)
 - Platforma kot storitev (ang. *Platform as a Service* - PaaS)
 - Infrastruktura kot storitev (ang. *Infrastructure as a Service* - IaaS)

National Institute of Standards and Technology, 2011

31

Software as a Service - SaaS

- Potrošnik uporablja aplikacije ponudnika, ki se izvajajo v oblaki infrastrukturi
 - Aplikacije so dostopne iz različnih uporabnikovih naprav prek vmesnika tankega odjemalca (npr. spletni odjemalec) ali programskih vmesnikov
- Potrošnik nima možnosti upravljanja/nadzora osnovne infrastrukture v oblaku (omrežje, strežniki, operacijski sistemi, shranjevanjem, itd.)
 - Uporabnikom je sicer lahko v izjemnih primerih omogočena konfiguracija aplikacij skozi nastavitve aplikacije

National Institute of Standards and Technology, 2011

32

Platform as a Service - PaaS

- Potrošniku je zagotovljena zmožnost, da na infrastrukturo oblaka namesti lastne aplikacije
 - ustvarjene z uporabo programskih jezikov, knjižnic, storitev in orodij, ki so podprti iz strani ponudnika
- Potrošnik nima možnosti upravljanja/nadzorovanja osnovne infrastrukture v oblaku (omrežje, strežniki, itd.)
 - ima pa nadzor nad nameščenimi aplikacijami ter možnost konfiguracije izvajalnega okolja, kjer se gostijo aplikacije

National Institute of Standards and Technology, 2011

33

Infrastructure as a Service - IaaS

- Potrošniku se zagotovijo storitve obdelave, shranjevanja, omrežne storitve ter drugi osnovni računalniški viri, ki omogočajo nameščanje in poganjanje programske opreme, ki lahko vključuje tudi operacijske sisteme in aplikacije
- Potrošnik nima možnosti upravljanja oz. nadzora podporne oblačne infrastrukture
 - lahko pa nadzira operacijske sisteme, shrambo podatkov ter nameščene aplikacije
 - običajno ima potrošnik tudi omejene možnosti izbire in nastavljanja omrežnih komponent (npr. požarni zid)

National Institute of Standards and Technology, 2011

34

Drugi storitveni modeli v računalništvu v oblaku

- Zaledje kot storitev (ang. Backend as a Service - BaaS)
- Funkcija kot storitev (ang. Function as a Service - FaaS)
- Aplikacija kot storitev (ang. Application as a Service - AaaS)
- Računska storitev kot storitev (ang. Computing as a Service - CaaS)
- Podatkovna shramba kot storitev (ang. Storage as a Service - STaaS)
- Poslovni proces kot storitev (ang. Business process as a Service - BPaaS)
- ...

35

Vrste oblakov

- Osnovne vrste oblakov
(National Institute of Standards and Technology, 2011):
 - Zasebni oblak (ang. private cloud)
 - Skupnostni oblak (ang. community cloud)
 - Javni oblak (ang. public cloud)
 - Hibridni oblak (ang. hybrid cloud)
- Ter nove vrste oblakov, npr.:
 - Multioblak (ang. multi-cloud)
 - Visoko-zmogljivi računski oblaki (ang. high-performance computing cloud)
 - Regijski oblak (ang. region cloud)
 - ...

36

Arhitektura za oblak

- Arhitektura za oblak (ang. cloud-native arhitektura) je nastala zaradi izzivov pri razvoju sodobnih rešitev
 - zahteva po vedno hitrejšem razvoju in dostavi rešitev
 - razvoj in vpeljava tehnoloških novosti
 - nenehne spremembe v organizacijah, na katere moramo biti sposobni hitro odreagirati in prilagoditi poslovne rešitve
 - ...
- Je arhitektura, ki vključuje
 - tehnološke novosti +
 - nove pristope k razvoju, testiranju, nameščanju, uporabi in vzdrževanju programskih rešitev

37

Arhitektura za oblak

- Celovita arhitektura, filozofija pristopa k gradnji aplikacij, ki v polni meri izkoriščajo računalništvo v oblaku.
- **Ni poudarek na strežnikih ampak na storitvah!**
- Cilj večine organizacij, ki so v procesu transformacije svojih poslovanj
 - Podjetja iščejo rešitve za transformacijo poslovanja - modernizacijo infrastrukture in procesov, pri čemer previdno izbirajo med oblačnimi tehnologijami, ki najbolj ustrezajo njihovim potrebam in so se sposobne hitro prilagoditi glede na spremembe v prihodnosti

38

Načela arhitekture za oblak

Kontejnerizacija

- enkapsulacija aplikacij skupaj z vsemi odvisnostmi v obliki paketa, ki ga lahko enostavno namestimo, poženemo in testiramo v poljubni platformi

Dinamično upravljanje

- uporaba javno-dostopnih oblčnih storitev, ki omogoča plačevanje samo glede na porabo

Mikrostoritve

- Razvoj rešitev na osnovi manjših, šibko sklopljenih storitev, ki jih lahko po potrebi zamenjamo, razvijemo na novo, itd.

Avtomatizacija

- čim več opravil, ki smo jih prej opravljali manualno, je potrebno zamenjati z avtomatiziranimi opravili

Orkestracija

- Zagotoviti je potrebno celovito upravljanje nameščanja in izvajanja storitev, ki so nameščene v obliki zabojnikov. Rešitve kot je Kubernetes omogočajo upravljanje in avtomatizacijo opravil nameščanja, zagotavljanja dosegljivosti, itd. storitev, nameščenih v obliki zabojnikov

Vir: P.Reznik, J.Dobson & M.Gienow, Cloud Native Transformation, Practical Patterns for Innovation, 2020

39

5 načel arhitekture za oblak

Načrtuj za avtomatizacijo

- Avtomatiziraj vzpostavitev infrastrukture, gradnjo in nameščanje rešitev, prilagajanje zmogljivosti, nadzor in samodejno odpravljanje napak.

Premišljeno ravnaj s stanjem

- Upravljanje stanja v porazdeljenih sistemih je zahtevno, zato naj bodo komponente praviloma **brez stanja**; stanje naj bo shranjeno v za to namenjenih storitvah.

Dajaj prednost upravljanim storitvam

- Kadar je mogoče, uporabi storitve, za katere vzdrževanje, posodobitve in razpoložljivost zagotavlja ponudnik oblaka.

Zagotavljal večplastno zaščito

- Sistem zaščiti na več ravneh: pred napadi od zunaj in zlorabami znotraj sistema (preverjanje identitete, omejevanje dostopa, nadzor prometa, beleženje).

Arhitekturo nenehno izpopolnjuj

- Oblačne rešitve se stalno spreminjajo, zato mora tudi arhitektura sproti slediti potrebam organizacije, obremenitvam in razvoju tehnologij.

Vir: <https://cloud.google.com/blog/products/application-development/5-principles-for-cloud-native-architecture-what-it-is-and-how-to-master-it>

40

7 načel arhitekture za oblak

Storitveno usmerjeno načrtovanje	• Sistem razdelimo na storitve z jasnimi mejami in dogovorjenimi vmesniki.
Prožnost	• Sistem se samodejno prilagaja obremenitvi (poveča ali zmanjša število virov po potrebi).
Zmožnost opazovanja	• Vgrajeni so zapisi, meritve in sledenje, da lahko hitro ugotovimo, kaj se dogaja in kje je težava.
Odpornost na napake	• Napake so pričakovane; sistem jih omeji, se po možnosti sam popravi in ostane uporaben.
Avtomatizacija vseh postopkov	• Samodejna vzpostavitev, nameščanje, nadgradnje, nadzor in odzivanje na težave.
Načelo ničelnega zaupanja	• Ne predpostavljamo zaupanja; vsak dostop se preveri, pravice so omejene na nujno potrebno.
Nenehni razvoj arhitekture	• Arhitekturo sproti prilagajamo potrebam, spremembam v organizaciji in razvoju tehnologij.

Vir: https://www.alibabacloud.com/blog/seven-principles-of-cloud-native-architecture_598431